Helios decomposition plan

Oct 29, 2026 · @Chris

Decisions

Keep one repo. Split helios into four packages inside the existing repository as a `uv` workspace, each with its own `pyproject.toml`, version, and test suite. Four repos would turn every cross-package change into four pull requests and a dependency-publish dance before anything runs; today the four packages are 1 commit old, share one author, and have no external consumers. Split the repo later, if ever, when a package gets a second team or an outside consumer (see Repository strategy).

Daedalus needs two small changes, not a redesign. Daedalus can federate the helios-serve MCP server today using its `mcp` connection protocol and a static bearer token, but it cannot forward the `X-Helios-Principal-Assertion` header, so every call would run as `HELIOS_MCP_DEFAULT_PRINCIPAL`. Fix on the helios side: let helios-serve accept the caller's JWT via `forward-token` and derive the principal from `jwt.sub`. Fix on the Daedalus side: a `forwarded_headers` field on `ConnectionSpec` (or a `principal-assertion` auth mode) so a signed assertion can ride along. Also implement `server/discover` in helios-serve, because Daedalus federation requires it (`supports_discover()`).

The talk-to-data prompt becomes a spec-conformant skill. The Agent Skills specification is an open format (a directory with `SKILL.md`, YAML frontmatter `name` + `description`, optional `license`, `compatibility`, `metadata`, `allowed-tools`); membership in the Agentic AI Foundation is not required to use it. Mistral Vibe states it follows the spec and discovers skills from `./.agents/skills/` and `~/.agents/skills/`, so a conformant `talk-to-data` skill runs there, in Claude Code, goose, and any other adopting host. The prompt text today lives inline in `apps/helios/console/conversation.py:320-345`; it moves to `skills/talk-to-data/SKILL.md` and the Python loads it from there (see the skill section).

Question	Answer	Why
One repo or four?	One repo, four packages (`uv` workspace)	No external consumers yet; cross-package refactors stay atomic; split later per package

Question	Answer	Why
Update Daedalus?	Yes: header forwarding + `server/discover` on the helios side	Federation works now only with a static token and a default principal
Agent Skills without AAIF?	Yes	Open spec on agentskills.io; Mistral Vibe, Claude Code, goose read it
First thing to do	Purge `.cursor/mcp.json` from history (throw-away token over TPC-DS data, so no exposure; still shouldn't ship), add LICENSE	Cheap to do now, expensive once the repo has consumers

Current state

Helios is ~63k lines of Python and ~28k of TypeScript in one commit (`316f5d8` , 2026-10-08), with `shared/helios_core` (~9.2k LOC) already shaped like a kernel and `apps/helios` holding the console, crawler, MCP server, and graph gateway. The dependency graph inside `helios_core` has no cycles, so the split is mostly moves rather than untangling.

Packaging is nominal, not real. Three `pyproject.toml` files exist (hatchling) but nothing installs: `shared/helios_core/pyproject.toml` sits inside the package and points at a non-existent `helios_core/helios_core` ; `apps/helios/pyproject.toml` declares `helios-core @ {path=...}` in Poetry syntax that pip rejects; `__version__` says 0.1.0 while the pyproject says 1.0.0. Code reaches `helios_core` through `PYTHONPATH` built by the Workbench launchers (`apps/helios/*/app.py`) and `sys.path.insert` in `jobs/*.py` and `scripts/*.py` . `apps/helios/mcp/__init__.py` shadows the MCP SDK's `mcp` package on pytest's path.

Where things live today

Area	Path	LOC (approx.)	Tests
Ontology (LinkML parse, graph, mapping)	`shared/helios_core/ontology`	1,014	68 (with index)

Area	Path	LOC (approx.)	Tests
Ossie model + SQL compiler	`shared/helios_core/ossie`, `compiler`	905	in core misc (92)
Index store, records, evidence (LanceDB)	`shared/helios_core/index`	1,762	shared with ontology
Operational metadata (SQLite, 16 tables, 8 migrations)	`shared/helios_core/metadata`	2,872	core misc
Identity, authz, delegation, audit, tracing	`shared/helios_core/authz`, `identity.py`, `delegation.py`, `audit.py`, `tracing.py`	~700	core misc
LLM client (Anthropic, OpenAI-compatible/CAII, Mistral)	`shared/helios_core/llm`	579	—
Harvest, profile, propose, Atlas	`shared/helios_core/catalog`, `profiler`, `propose`, `atlas`	701	**none**
Crawler (connectors, mentions, resolution, claims)	`apps/helios/crawler` (+ `helios_core/crawler` settings)	7,077 + 722	177
MCP server (9 tools)	`apps/helios/mcp/server.py`	~1,200	in console/serve (141)
Console API + conversation + assistant + evaluation	`apps/helios/console` (`api.py` alone ~4k)	~8k	141
Graph gateway (Memgraph)	`apps/helios/graph`	—	67
Jobs (Workbench entry points)	`jobs/{harvest,profile,propose,publish,crawl}.py`	small	none

Three places core already reaches into feature code, which the split must cut: `index/crawler_settings.py` imports `helios_core.crawler.settings`; `health.py` imports `atlas`; `publication.py` and `graph.py` import `review` (publish-workflow code).

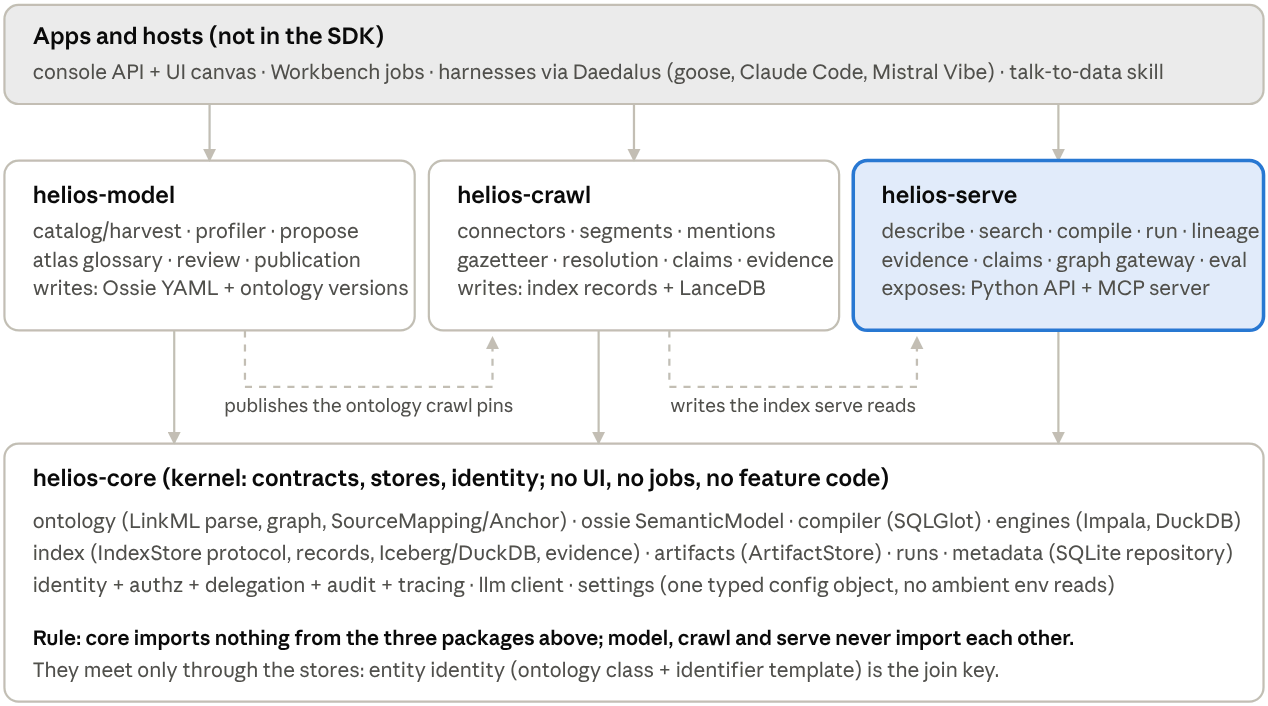
Config is read ambiently in `runs.py` (at import time), `artifacts.py`, `metadata/sqlite.py`, `llm/client.py`, `index/evidence.py`, `audit.py`, plus `config.py`'s `atlas_config()` / `impala_config()` with ~12 callers across core, console, crawler, MCP, and jobs. Repo-root paths are hardcoded via `Path(__file__).parents[3]` in `console/ontology.py`, `console/assistant.py`, `crawler/__main__.py`, and `console/evaluation.py` uses `parents[2]`, which resolves `BUILTIN_SUITE` to `apps/eval/tpcds-v1.json` instead of `eval/tpcds-v1.json` (a latent bug).

Hygiene to clear before anything ships: no LICENSE; CI only for `helios-ds-generation`; a throw-away bearer token in `.cursor/mcp.json`; committed `runs/` (1.0 MB), `run_archive/` (492 KB), `docs/Helios-Overview.pptx` (1.8 MB), `apps/helios/ui/dist/`, `.DS_Store`, `review.htmnl`, a root `impala_check.py` that is a diff, and a stale root `compiler/` with `__init_.py`.

Target architecture

Four installable packages, one dependency direction: `helios-model`, `helios-crawl`, and `helios-serve` each depend on `helios-core` and never on each other. Composition happens through the stores, not imports: a crawl run pins an `ontology_version`, resolved mentions carry entity identifiers built from the ontology's `SourceMapping` / `Anchor` templates, and serve joins structured rows and unstructured evidence on those identifiers.

Everything depends on helios-core; only helios-serve reads both stores



target package layout · 4 packages, one dependency direction

Solid arrows read "depends on" (imports); dashed arrows are data flows through the stores, never imports.

Package contents and public contracts

Package	Moves in (from today)	Public API (stable surface)	Must not contain
`helios-core`	`ontology`, `ossie`, `compiler`, `engines`, `index` (minus `crawler_settings.py`), `artifacts`, `runs`, `metadata`, `authz`, `identity`, `delegation`, `audit`, `tracing`, `llm`, `domain`, `config` (rewritten as `settings`)	Pydantic contracts (`SemanticModel`, `OntologyGraph`, `SourceMapping`, `*Record`), protocols `IndexStore`, `ArtifactStore`, `Engine`, `MetadataRepository`; `Principal`; `Settings`; `LLMClient`	`health.py` (moves to serve), `publication` / `review` / `graph.py` (move to model), `crawler/` (moves to crawl), any `os.environ` read outside `settings`
`helios-model`	`catalog`, `profiler`, `propose`, `atlas`, `publication`, `review`, `graph.py`; `jobs/{harvest,profile,propose,publish}.py` become CLI entry points	`Catalog.harvest(source, principal)`, `Profiler.profile(...)`, `Proposer.propose(...) -> Proposals`, `Publisher.publish(proposals, principal) -> OntologyVersion`; proposals carry confidence + rationale	LLM provider wiring (uses core `LLMClient`), HTTP routes, UI

Package	Moves in (from today)	Public API (stable surface)	Must not contain
`helios-crawl`	`apps/helios/crawler/*`, `helios_core/crawler/{settings,sources,presets}`, `index/crawler_settings.py`, `index/evidence.embed_run`	`Crawler.run(source, ontology_version, principal) -> CrawlRun`, `Resolver.resolve(mentions, mapping)`, `Claims.extract(...)`, `Evidence.embed(run)`; connector protocol for new sources	The console's crawl routes, tryout/evaluate UI glue (evaluate stays as a CLI)
`helios-serve`	`apps/helios/mcp/server.py` tools, `health.py`, `apps/helios/graph/*`, `console/evaluation.py` (as a library + CLI), `index/evidence.search`	`Catalog.describe(ref)`, `search_semantics(q)`, `compile_query(req) -> SQL`, `run_query(req, principal)`, `explain_lineage(ref)`, `search_evidence(q)`, `entity_claims(id)`; `mcp_app(settings) -> ASGI`; `server/discover`	The agent loop (`ConversationService`), prompts, model-provider settings

Boundary rules enforced in CI (one `import-linter` contract each): core imports no sibling package; model/crawl/serve import only core and their own package; nothing imports `apps.*`; `os.environ` appears only in `helios_core/settings.py`.

What leaves the SDK: `console/api.py` (4k LOC) becomes thin FastAPI routers mounted per package; `ConversationService`, `assistant.py`, and `model_provider.py` move to an `apps/console` that is a *client* of the four packages and will later be replaced by a harness + the `talk-to-data` skill; the React UI stays in `apps/ui`; `helios-ds-generation` and `-evaluation` stay as separate test tooling.

Repository strategy

One repo, laid out as a `uv` workspace with four members, replaces today's three non-installable `pyproject.toml` files. Each package keeps its own version and changelog; one lockfile at the root pins the whole tree, so CI tests the exact combination a release ships.

```
helios/
├── pyproject.toml            # [tool.uv.workspace] members = ["packages/*",
"apps/*"]
├── uv.lock
├── packages/
|   ├── helios-core/   pyproject.toml, src/helios_core/, tests/
|   ├── helios-model/  pyproject.toml, src/helios_model/, tests/
|   ├── helios-crawl/  pyproject.toml, src/helios_crawl/, tests/
|   └── helios-serve/  pyproject.toml, src/helios_serve/, tests/
├── apps/
|   ├── console/       FastAPI app + Workbench launchers (depends on all
four)
|   ├── ui/            React canvas
|   ├── helios-ds-generation/   (unchanged)
|   └── helios-ds-evaluation/   (unchanged)
├── skills/
|   └── talk-to-data/  SKILL.md (+ references/)
├── ontology/  models/  eval/        # data, versioned, not Python
└── runtime/   Dockerfile + build.sh
```

Why not four repos now

Concern	One repo (workspace)	Four repos
A change that touches a core contract and its consumers	One PR, one CI run, atomic	Publish core, bump three pins, three PRs; broken window in between
Dependency pins	`uv.lock` pins everything; path deps resolve locally	Each consumer pins a core version; drift is the default
Tests that span boundaries (`test_talk_to_data`, `test_graph_index`, `test_evidence`)	Stay as integration tests at the root	Must be rewritten against published wheels or deleted
Release cadence	Per-package version tags (`helios-core/v1.1.0`) from one repo	Independent, but nobody needs that yet
Who else consumes a package today	Nobody outside this repo	Nobody

Split a package out later when one of these becomes true: a second team owns it; an outside project depends on it and wants its own release cadence; or Daedalus/Blueprints wants to vendor `helios-serve` without the rest. `git subtree split packages/helios-serve` preserves history when that day comes.

Versioning and publishing. Start all four at `0.1.0` (`helios-core` drops its fictional 1.0.0). Core's contracts get a `CONTRACTS.md` that lists the public names; a change to that list is a minor bump. Publish wheels to the internal index from CI on tag; the Workbench runtime image installs them instead of building `PYTHONPATH`.

CI (one workflow, matrix over packages): `uv sync --all-packages`; `ruff` + `mypy` per package; `pytest packages/<pkg>/tests` per package with DuckDB-backed index fixtures; `import-linter` for the boundary contracts; root `tests/integration` for the cross-package suites, run on PR against a DuckDB + SQLite stack and nightly against Impala. Add a `LICENSE` (Apache-2.0 is the obvious choice given Ossie, LinkML, and the AAIF ecosystem) and a `CODEOWNERS` with one line per package.

Migration plan

Six phases, each leaving the app runnable and the 545 existing tests green. Every phase is a move plus one contract change; nothing is rewritten for its own sake. Estimates assume one engineer with the codebase in their head.

Phase 0 — Hygiene and a real build (2–3 days)

1. Remove `.cursor/mcp.json` with `git filter-repo` so it leaves history (the committed token is a throw-away over the TPC-DS environment, so this is hygiene, not an incident); add `.cursor/`, `runs/`, `run_archive/`, `.DS_Store`, `*.pptx` to `.gitignore`; delete the committed `runs/`, `run_archive/`, `ui/dist/`, root `impala_check.py`, root `compiler/`, `review.htmnl`.
2. Add `LICENSE` and a root `pyproject.toml` with `[tool.uv.workspace]`; move `shared/helios_core` to `packages/helios-core/src/helios_core` with a correct `pyproject.toml` (`packages = ["src/helios_core"]`); make `__version__` read from package metadata.
3. Rename `apps/helios/mcp` to `apps/helios/mcp_server` so the MCP SDK's `mcp` package stops being shadowed.

4. Replace every `sys.path.insert` and launcher-built `PYTHONPATH` with `uv sync` + editable installs; keep one `HELIOS_PYTHON_DEPS` shim for Workbench until the runtime image installs wheels.
5. CI: one workflow running `uv sync`, `ruff`, `pytest` for the whole tree. Exit: `uv run pytest` passes from a clean clone with no path hacks.

Phase 1 — helios-core becomes a kernel (1 week)

1. Cut the three core→feature edges: move `index/crawler_settings.py` out (to crawl, Phase 3); move `health.py` out (to serve, Phase 4); move `publication.py`, `review.py`, `graph.py` out (to model, Phase 2). Until those phases land, they live under `packages/helios-core/src/helios_core/_pending/` and an `import-linter` contract forbids new edges.
2. Replace ambient config with one `Settings` object (pydantic-settings) in `helios_core/settings.py`: fold `config.atlas_config()`, `impala_config()`, and the env reads in `runs.py`, `artifacts.py`, `metadata/sqlite.py`, `llm/client.py`, `index/evidence.py`, `audit.py`. `runs.py` stops reading env at import time. Every store and engine takes `Settings` (or its slice) in its constructor.
3. Replace `Path(__file__).parents[n]` with `Settings.repo_root` / `Settings.models_dir` / `Settings.ontology_dir`; this also fixes the `BUILTIN_SUITE` path bug in `evaluation.py`.
4. Write `CONTRACTS.md` listing the public names; everything else gets a leading underscore or moves under `_internal/`.
5. Tests: the 92 "core misc" and 68 index/ontology tests move to `packages/helios-core/tests` with the `persistent_auth_stack` fixture. Exit: `helios-core` installs standalone; `import helios_core` triggers no env read and no file I/O.

Phase 2 — helios-model (1 week)

1. Move `catalog`, `profiler`, `propose`, `atlas`, plus `publication`, `review`, `graph.py` from `_pending/`. Jobs `harvest/profile/propose/publish.py` become `helios-model` console-script entry points (`helios-model harvest --source ...`) that call the library; `jobs/_common.py` dissolves into `Settings`.
2. Define the four public classes (`Catalog`, `Profiler`, `Proposer`, `Publisher`) as thin facades over the existing functions; `Proposals` is the typed object the UI canvas renders.

3. Console: `console/review.py` and the model-management routes in `api.py` become `apps/console/routers/model.py`, calling the facades.
4. Tests: there are none for harvest/profile/propose today. Write the minimum: a DuckDB-backed harvest→profile→propose→publish round-trip on the `helios-ds` tiny fixture, asserting the published Ossie YAML and `OntologyVersionRecord`. Exit: the round-trip test passes with no console import.

Phase 3 — helios-crawl (1.5 weeks; largest move)

1. Move `apps/helios/crawler/*` (7k LOC) to `packages/helios-crawl/src/helios_crawl`, plus `helios_core/crawler/{settings,sources,presets}` and `index/crawler_settings.py`. `index/evidence.py` splits: `embed_run` goes to crawl, `search` stays in core (serve calls it).
2. `crawler/__main__.py` becomes `helios-crawl run` with explicit `--ontology-version`; the console's `crawler.py`, `data_sources.py`, and `crawl_jobs.py` routes become `apps/console/routers/crawl.py`.
3. Connector protocol (`connectors/base.py`) becomes the documented extension point; `helios_ds`, `object_store`, `table_rows` ship as built-ins.
4. Tests: the 177 crawler tests and `test_evidence` move with it, including `crawler_samples.py`, `shapes.py`, `second_domain.py`, baselines. Untangle the test-module-imports-test-module pattern (`test_graph_index` importing `test_crawler_evaluate`) by promoting shared helpers to `helios_crawl.testing`. Exit: `helios-crawl` tests pass with DuckDB index and no console or graph import.

Phase 4 — helios-serve (1 week)

1. Move the nine MCP tools out of `mcp_server/server.py` into `helios_serve/api.py` as plain functions taking `(request, principal, stores)`; `server.py` shrinks to an ASGI factory `mcp_app(settings)` that registers them. Add `server/discover` for Daedalus federation.
2. Move `health.py`, `apps/helios/graph/*` (Memgraph gateway), and `console/evaluation.py` (library + `helios-serve eval` CLI) in.
3. Auth: keep `X-Helios-Principal-Assertion`; add a second path that accepts a JWT (issuer/JWKS from `Settings`) and builds the `Principal` from `sub` and claims, so a gateway can forward the caller's token (see Daedalus section).

4. Tests: the 67 graph tests and the serve half of the 141 console tests; `test_talk_to_data` (38) becomes a root integration test. Exit: `helios-serve` MCP server starts with `uvicorn helios_serve.mcp:app`, answers `tools/list` and `server/discover`, and passes its suite with no console import.

Phase 5 — Apps re-wired, agent loop extracted (1 week)

1. `apps/console` = routers from Phases 2–4 + `main.py` + `trace_documents`, `journeys`, `mcp_settings`, `model_provider`; `api.py` is deleted once its last route has a router.
2. `ConversationService` and `assistant.py` move to `apps/console/agent/`, marked as the legacy loop. Its system prompt becomes `skills/talk-to-data/SKILL.md`; `conversation.py` loads the body at startup so there is one source of truth.
3. Workbench launchers (`app.py` files) install wheels from the workspace instead of assembling `PYTHONPATH`.
4. Exit: console works end-to-end against the four packages; `apps/helios/` directory is gone.

Phase 6 — Retirement (ongoing)

Once a harness (goose, Claude Code, Mistral Vibe) runs the `talk-to-data` skill against `helios-serve` through Daedalus, delete `apps/console/agent/`, `model_provider.py`, and the conversation tables in SQLite (`conversations`, `conversation_messages`, `conversation_turns`); traces move to OpenTelemetry. `TraceRecorder` stays until then.

Phase	Weeks	Gate
0 Hygiene + build	0.5	Clean-clone `uv run pytest` green; token file purged
1 Core kernel	1	Standalone install; no env read on import; `CONTRACTS.md`
2 Model	1	Harvest→publish round-trip test
3 Crawl	1.5	177 tests green on DuckDB, no console import
4 Serve	1	MCP `tools/list` + `server/discover`; JWT auth path
5 Apps	1	`apps/helios/` deleted; skill is the prompt source

Phase	Weeks	Gate
6 Retire	—	A harness runs talk-to-data via Daedalus

Daedalus integration

Daedalus can already front the helios-serve MCP server, but only with a static token and a default principal, so per-user `doAs` and audit attribution are lost. Three changes close the gap: two in helios-serve, one in Daedalus.

What works today. `ConnectionSpec(protocol="mcp", federate=True)` adds every upstream tool to a Daedalus server as a gateway `mcp` target; `ToolSpec(kind="mcp", upstream_tool=...)` imports one tool through the loopback executor. Discovery runs `mcp_client.discover()` (protocol `2026-07-28` first, then `initialize` fallback) and snapshots the tool list into the release. Auth mode `static` sends `Authorization: Bearer <FACTORY_SECRET_*>`, which matches helios-serve's `HELIOS_MCP_TOKEN` check.

What breaks. helios-serve identifies the caller from `X-Helios-Principal-Assertion` (HMAC-SHA256, 60-second lifetime, issued by `delegation.issue_assertion`). Daedalus's gateway policies set only the `Authorization` header (`target_policies` in `gateway.py`); the executor adds `x-daedalus-user` and `x-daedalus-caller-token`, but not Helios's header. So every federated call lands on `HELIOS_MCP_DEFAULT_PRINCIPAL`, `run_query` runs `doAs` as that principal, and audit rows name it. Daedalus federation also requires `server/discover` (`supports_discover()`), which helios-serve does not implement.

#	Change	Where	Effort
1	Implement `server/discover` (name, description, input schema, `annotations.readOnlyHint` per tool; read-only for everything except `run_query`)	helios-serve, Phase 4	0.5 day
2	Accept a JWT as an alternative to the assertion: `Settings.mcp_jwt_issuer` + JWKS; build `Principal` from `sub`, `org`, and a `helios_roles` claim; keep the assertion path for the legacy console	helios-serve, Phase 4	1–2 days

#	Change	Where	Effort
3	`ConnectionSpec.forward_headers: list[str]` (allow-listed request headers copied to the backend) or a new `auth_mode="principal-assertion"` where Daedalus mints the Helios assertion from `jwt.sub` with a shared secret ref	Daedalus `contracts.py`, `gateway.py`, `executor.py`, one migration	2 days

Change 2 is the one that matters: with `FACTORY_JWT_ISSUER` set, Daedalus's `forward-token` mode already passes the caller's own JWT to the backend. If helios-serve can read that JWT, the chain user → harness → agentgateway → helios-serve → Impala `doAs` works with no new Daedalus code. Change 3 is the fallback for deployments without an OIDC issuer.

Packaging helios-serve for Daedalus. Ship it as a Workbench application from the `helios-serve` wheel (`uvicorn helios_serve.mcp:app`), registered in Daedalus as one `mcp` connection. Two publication shapes: a federated server ("everything Helios exposes") for talk-to-data, and a curated server that imports only `search_semantics`, `describe`, `compile_query` for harnesses that should read metadata but never execute SQL. Daedalus's `effect` field is metadata only today, so the second shape is how least-privilege is enforced until per-tool authorization exists.

Not needed: rewriting helios-serve tools as Daedalus Python packs. Packs are for small curated tools over REST; the Helios tools carry authorization, tracing, and store access that belong in the package, and federation keeps one implementation.

The talk-to-data skill

The prompt in `conversation.py:320-345` is already most of a skill; wrapping it in the Agent Skills format takes an afternoon and makes it runnable in Mistral Vibe, Claude Code, goose, and anything else that reads `SKILL.md`. AAIF membership is not involved: the specification is public, and Mistral's docs state that Vibe "follows the Agent Skills specification" and loads skills from `./.agents/skills/` (project) and `~/.agents/skills/` (user), with `/reload` picking up changes (Mistral Vibe skills docs).

Conformance checklist (from the spec): directory name equals `name`; `name` is lowercase letters, digits, hyphens, max 64, no leading/trailing/double hyphens; `description` 1–1024 characters stating what and when; optional `license`, `compatibility` (≤500 chars),

`metadata` (string→string), `allowed-tools` (space-separated, experimental; Vibe reads it but does not enforce it); body under 500 lines; references one level deep in `references/`; validate with `skills-ref validate ./talk-to-data`.

Three changes to the prompt itself. (1) Drop the `{model_id!r}` interpolation and "never request another model" — a skill is static text and the host picks the model; model pinning belongs in the harness or gateway config. (2) Name the MCP server the skill expects (`helios`) so tool references resolve in hosts that prefix tools by server. (3) Move the TPC-DS key example into `references/` so the body stays generic across semantic models.

```
---
name: talk-to-data
description: Answer questions over a Cloudera lakehouse through the Helios
MCP tools — governed semantic metrics (search_semantics, describe,
compile_query, run_query), lineage, and crawled documents (search_evidence,
entity_claims). Use when the user asks about warehouse data, metrics, what
customers or documents say, or wants figures joined to document evidence.
license: Apache-2.0
compatibility: Requires the Helios MCP server (helios-serve) registered as an
MCP server named "helios".
metadata:
  author: cloudera
  version: "1.0"
  helios-prompt-version: talk-to-data-v1
allowed-tools: mcp__helios__search_semantics mcp__helios__describe
mcp__helios__compile_query mcp__helios__run_query
mcp__helios__explain_lineage mcp__helios__search_evidence
mcp__helios__entity_claims
---

# Talk to your data (Helios)

Use only the Helios MCP tools for semantic metadata, compilation, lineage,
and structured or unstructured data. Do not invent data.

## Querying structured data (warehouse)
1. Before compiling or running a query, call `search_semantics` and copy
metric names exactly from its results.
2. Dimensions and filter columns must use exact `database.table.column`
identifiers returned by `search_semantics` or `describe`. Never invent,
shorten, or convert semantic names.
```

```
3. Preserve datatypes: JSON numbers for numeric, booleans for Boolean, ISO
strings for Date/Timestamp.
4. If the tool reports an invalid query or filter value, search again and
correct identifiers and JSON types.

## Crawled documents (emails, chats, reports)
- For what people wrote or said (complaints, reasons, requests), call
`search_evidence` with the user's question in their own words; it returns
passages by meaning, one per document. Answer from the passages, quoting what
was written.
- For what documents claim about one entity (a customer, item, store, sale,
return), call `entity_claims`.
- Quote or paraphrase only passages the tools returned, and say which
document type they came from.

## Combining documents and tables
Each passage lists the entities its document is linked to, with warehouse
keys. To add figures from the warehouse, use those key values as filters in
`run_query` after finding the field with `search_semantics`. A passage with
no linked entity cannot be joined to a table; say so. See [references/entity-
keys.md](references/entity-keys.md) for the key format.

## General rules
- Never label semantic validation, authorization, or tool-argument errors as
a warehouse outage.
- If a tool reports that it is unavailable, tell the user plainly.
```

Layout in the repo: `skills/talk-to-data/SKILL.md`, `skills/talk-to-data/references/entity-keys.md` (the `tpcds.customer:c_customer_sk=12345` example and the identifier-template rules), later `skills/talk-to-data/references/semantic-models.md` generated from published Ossie models. `conversation.py` reads the body from this file in Phase 5 so the legacy console and every harness run the same text.

Running it in Mistral Vibe: copy or symlink `skills/talk-to-data` into `./.agents/skills/` (or add the repo path to `skill_paths` in `config.toml`), register helios-serve as an MCP server named `helios` through a Vibe plugin (MCP servers are configured via plugins per the docs), then `/talk-to-data` or let the agent load it on its own. The same directory works unchanged in Claude Code (`.claude/skills/` or `.agents/skills/`) and goose.

Risks and open questions

The plan's main risk is scope creep inside Phase 1: the `Settings` refactor touches every module, and it is tempting to redesign stores while there. Hold the line at "same behaviour, injected config"; store redesign is a separate project.

Risk	Likelihood	Mitigation
Workbench runtime still needs `PYTHONPATH` because wheels aren't published yet	High	Keep the `HELIOS_PYTHON_DEPS` shim through Phase 5; publish to the internal index in Phase 0's CI even if nothing consumes it yet
Crawler tests that import other test modules break when moved	High	Promote shared helpers to `helios_crawl.testing` in Phase 3 before moving files
`index/evidence.py` split (embed in crawl, search in core/serve) leaves LanceDB schema ownership unclear	Medium	Core owns the table schema and the `search` reader; crawl owns `embed_run`; a schema-version column guards mismatch
JWT path in helios-serve diverges from the assertion path (two principals, two audit shapes)	Medium	Both paths produce the same `Principal`; one test asserts identical audit rows for the same user via either path
Daedalus snapshots the tool list per release, so a new helios-serve tool is invisible until re-published	Low	Document it; it is the intended behaviour of immutable releases
`helios-model` ships with zero tests today	Certain	The Phase 2 round-trip test is the gate; no facade without it
Ossie upstream (`/opt/ossie`, `OSSIE_HOME`) is cloned at image build with no pin	Medium	Pin a commit in `runtime/Dockerfile` and record it in `compatibility.yaml` as Daedalus does for agentgateway

Open questions

- [ ] Which OIDC issuer will front CAI Workbench apps in the target deployment? The JWT path in helios-serve needs an issuer and JWKS URL to be useful; without one, Daedalus change 3 (assertion minting) is required instead.

- [ ] Does Memgraph stay? The graph gateway is 67 tests and a separate runtime for what is, today, a disposable cache. If `helios-serve` can answer `explain_lineage` and neighbourhood queries from the Iceberg index, drop it.
- [ ] Where do traces go after the console's `TraceRecorder` retires: OpenTelemetry to the customer's collector (matching Daedalus's OTel dependency), or stay in SQLite?
- [ ] License: Apache-2.0 for all four packages, or does Cloudera want helios-serve under a different grant from the SDK?
- [ ] `helios-ds-generation` has its own `impala.py` that imports core `impala_config()`; after Phase 1 it should depend on `helios-core` properly or copy the 40 lines.

First two weeks

1. Day 1: purge `.cursor/mcp.json`, add LICENSE, open the hygiene PR.
2. Days 2–3: root `uv` workspace, `helios-core` installable, `mcp` package rename, CI green on a clean clone.
3. Days 4–8: Phase 1 — `Settings`, cut the three core→feature edges, `CONTRACTS.md`, `import-linter`.
4. Days 9–10: write `skills/talk-to-data/SKILL.md`, validate with `skills-ref`, run it in Mistral Vibe against the existing MCP server (static token is fine for this test) to prove the skill works before any serve refactor.

That last step is deliberately early: it shows the end state (a third-party harness driving Helios through a standard skill and MCP) in week two, with the SDK refactor still ahead.